Expected Linear Time Algorithm for Computing 3D Relative Neighborhood Graphs

Z. Zzyzek[a,*,1] (Researcher)

ARTICLE INFO

Keywords:
graph algorithm
relative neighborhood graph
computation geometry
RNG

ABSTRACT

An expected linear time algorithm ($O(n)$) for computing the relative neighborhood graph (RNG) for points in Euclidean space, distributed uniformly in a unit three dimensional cube is given. To the author's knowledge, this improves on the previously best known $O(n \log(n))$ in three dimensions. The algorithm considers an individual anchor point and limits a volume that potential RNG neighbors can be in through a plane partition heuristic. The K surrounding potential neighbors are then tested with a naive $O(K^3)$ algorithm. The RNG has finite degree when considering random point distributions in three dimensions, resulting in an expected finite K, making $O(K^3) = O(1)$, providing the linear expected time algorithm when enumerating through all points.

1. Introduction

Let $P = (p_0, p_1, \ldots, p_{n-1})$ be a set of points in $\mathbb{R}^3$. The relative neighborhood graph, RNG(P), is a graph that connects two vertices, p_i, p_j if and only if there is no other vertex within the *lune* between them.

$$(p_i, p_j) \in \mathrm{RNG}(P) \iff |p_i - p_j| \leq \max_{k \neq i,j}(|p_i - p_k|, |p_j - p_k|)$$

Figure **??** shows a graphical representation of points connected ....

2. Related Work

3. Motivation and Structure

For $P = (p_0, p_1, \ldots, p_{n-1})$, a brief overview of the algorithm roughly proceeds as follows:

- Create a grid, where each cell size of $\frac{1}{n^{1/3}}$
- Bin points into grid cells, allowing for multiple points in a bin
- For each point, p (called an *anchor point*):
  - keep adding neighbor points, q, from nearby grid cells and discard further away points whose connection to p would be sabotaged by the existence of q
  - keep extending the radius of grid cells and collecting potential q neighbors until all other points outside the currently considered grid cells are guaranteed to not be able to connect to p
  - add nearby additional points as needed and run a naive relative neighborhood graph calculation for p on the pool of q neighbors collected

A coarse grained plane partition heuristic is used to exclude further away points from consideration.

Given points p and q, any point, w, that lies on the other side of the plane with normal $\frac{(q-p)}{|q-p|}$ that passes through q, cannot connect to p, as q would be in any lune that p and w would create. Figure **??** gives a graphical representation of points excluded by this plane heuristic in two dimensions.

The plane partition heuristic is coarse, as the entire region that q excludes from p is more complicated (see Figure **??**). While the plane partition heuristic is coarse, it's easy to caclulate and provides a good enough way to exclude large portions of points so that it can be used as the foundation for an expected linear time algorithm.

To maintain a linear time algorithm, we need to avoid processing points individually and to implicitely exclude large collection of points that can never connect to the our anchor point p. To do this, we consider a current grid cell collection of integral radius $R \in \mathbb{Z}$, centered on the grid cell where anchor point p resides. If the tangent planes that passes through neighbor q, and in direction $(q - p)$ within the grid cell collection, partition each of the six sides of our cube of radius R, we can be assured that any points outside of the grid cell collection will never be able to connect to p.

A grid cell collection of a radius R is called a *fence* of radius $R \in \mathbb{Z}$. When R is understood from context, this will be called just a *fence*. If the sides of the *fence* has been partitioned, with anchor point p and neighbors q within the *fence*, the *fence* will be called *secured*.

Points outside of the *fence* will never be able to connect to anchor point p but they still might be in the lune of some points within the *fence*. For this reason, when collecting points for the naive relative neighborhood graph calculation, some points outside of the fence will need to be added, as they can sabotage potential edges within the *fence*.

This will be done by considering a larger radius of the current *fence* to add points but the increase in R will shown to be bounded by a constant factor, allowing overall linear time to be maintained.

Before discussing the algorithm in detail, preliminary structural results will be discussed.

zzyzek@thumpernet.com (Z. Zzyzek)
zzyzek.github.io (Z. Zzyzek)
ORCID(s): 0009-0005-2175-1166 (Z. Zzyzek)

If w is on the other side of the plane that passes through q and with tangent in the direction of $(q - p)$, then q will always be in the lune created by p and w (see Figure **??**):

Lemma 1 (Plane Partition Heuristic).

$$p, q, w \in \mathbb{R}^3$$

if

$$w \in H^+_{p,q} = \{u \ : \ (q - p) \cdot (u - q) \geq 0\}$$

then

$$q \in L_{p,w} = \{v \ : \ |v - p| \leq |p - w| \ \& \ |v - w| \leq |p - w|\}$$

.

Only points within the convex hull need to be considered for a relative neighborhood graph connection to p:

Theorem 2. *Let $p \in \mathbb{R}^3$, $Q = (q_0, q_1, \ldots, q_{m-1}) \subset P$ and C be the compact convex hull created from $H_i^- = \{u \in \mathbb{R}^3 | (q_i - p) \cdot (u - q_i) < 0\}$, then $w \notin C \rightarrow w \notin RNG(P, p)$*

Proof. All w outside of C will have some $q_i \in Q$ in the $L(w, p)$ lune, so cannot have an RNG edge to p.

4. Algorithm

5. Conclusion

6. Bibliography

A. Appendix.0

Appendix sections are coded under `\appendix`.

B. Appendix.1

... Appendix sections are coded under `\appendix`.

...